SEMINÁRIO DE BANCOS DE DADOS · GRUPO 5

Bancos multimodelo com ArangoDB

Documento, grafo e chave-valor no mesmo motor, consultados com uma linguagem só.

documento grafo chave-valor

Lucas Beno · João Henrique · Arthur Oliveira · Arthur Nogueira · Gustavo

PARTE 1 · TEORIA DO MODELO

Os quatro modelos NoSQL

Cada modelo é especialista em um tipo de dado.

Documento

Dados em JSON, com esquema flexível.

`ex.: MongoDB`

Chave-valor

Chave → valor, leitura e escrita muito rápidas; cache e sessões.

`ex.: Redis`

Grafo

Nós e arestas, com foco nos relacionamentos.

`ex.: Neo4j`

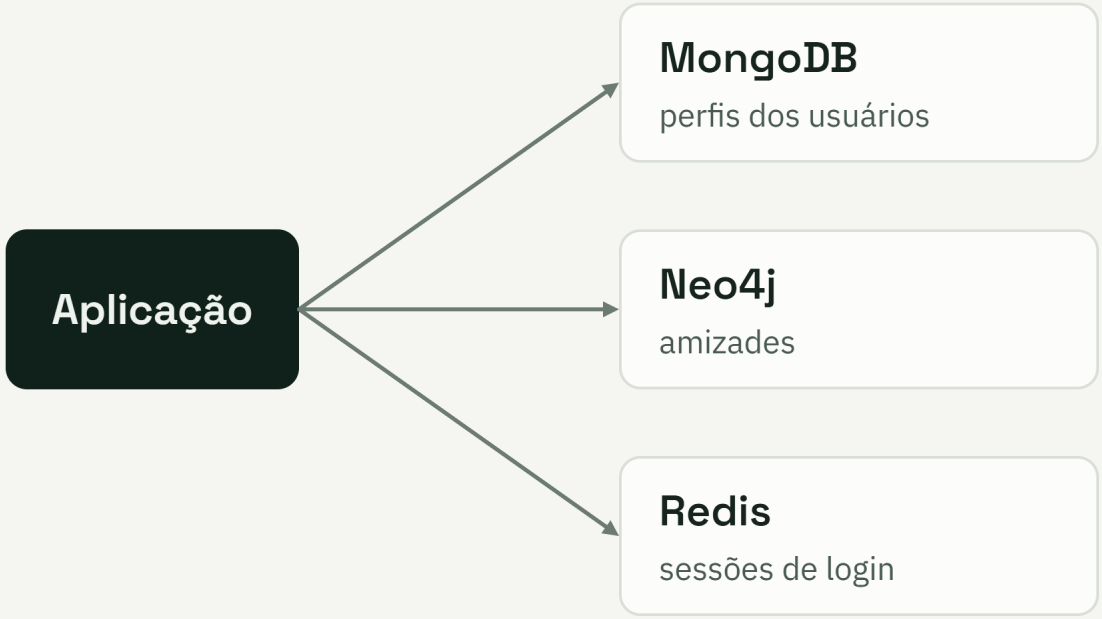
Colunas

Dados divididos em partições, para grande volume de escrita.

`ex.: Cassandra`

O problema: persistência poliglota

Um banco diferente para cada tipo de dado, termo popularizado por Martin Fowler.

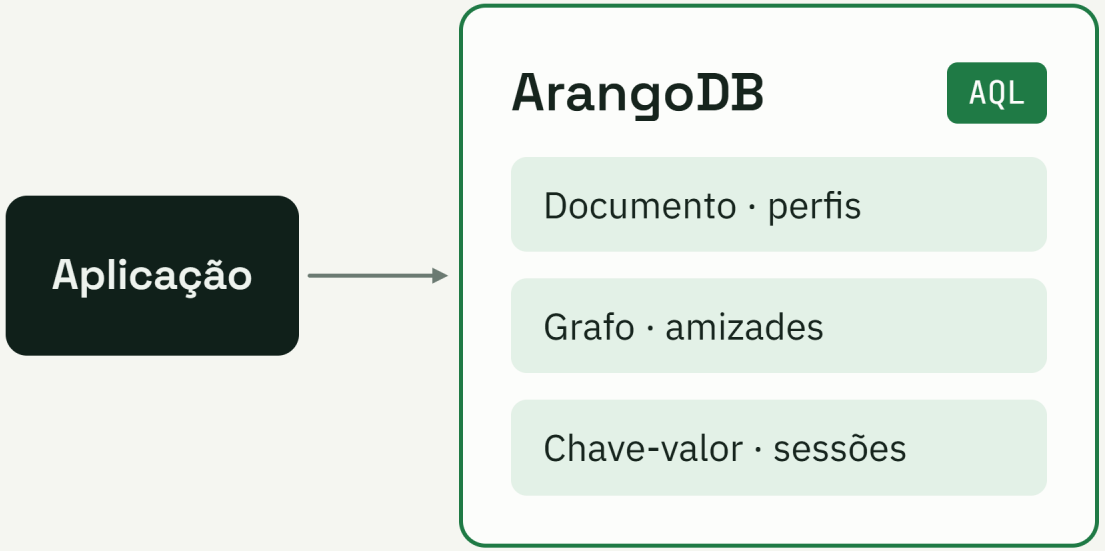


O custo operacional

- Vários servidores, cada um com backup, segurança e monitoramento
- Uma linguagem de consulta para cada banco
- Dados duplicados que precisam ficar sincronizados
- Consultas entre modelos montadas no código da aplicação

A solução: um banco multimodelo

Um único motor que suporta vários modelos de forma nativa, com uma linguagem e uma infraestrutura só.



Como um celular

Câmera, GPS e música num aparelho só, em vez de três aparelhos separados.

No nosso projeto, tudo fica no ArangoDB: **usuarios** (documento), **amizades** (grafo) e **sessoes** (chave-valor).

Bancos multimodelo no mercado

Banco	Modelos	Como funciona
ArangoDB	Documento, grafo e chave-valor	Multimodelo nativo, consultado com AQL
Azure Cosmos DB	Documento, chave-valor, grafo e colunas	Uma API diferente para cada modelo
Couchbase	Documento e chave-valor	Os dois modelos no mesmo motor
PostgreSQL + JSONB	Relacional e documento	Relacional que ganhou suporte a JSON

Escolhemos o ArangoDB porque o grafo é nativo: no PostgreSQL, “amigos de amigos” exigiria CTE recursiva ou extensão.

PARTE 2 · VANTAGENS E DESVANTAGENS

Por que escolher um multimodelo?

Uma linguagem só

Consultas entre modelos cabem numa query AQL, sem juntar resultados no código.

Uma infraestrutura só

1 contêiner Docker em vez de MongoDB + Redis + Neo4j.

Consistência

Um backend só, sem atraso de sincronização, com transações ACID.

Menor custo total

Uma licença, um fornecedor e um conhecimento para a equipe.

Esquema flexível

Documentos JSON sem esquema fixo: cada perfil pode ter campos diferentes.

Persistência poliglota, sem manter vários SGBDs.

PARTE 2 · VANTAGENS E DESVANTAGENS

Os limites

O preço da generalização é não ser o melhor em nenhum modelo isolado.

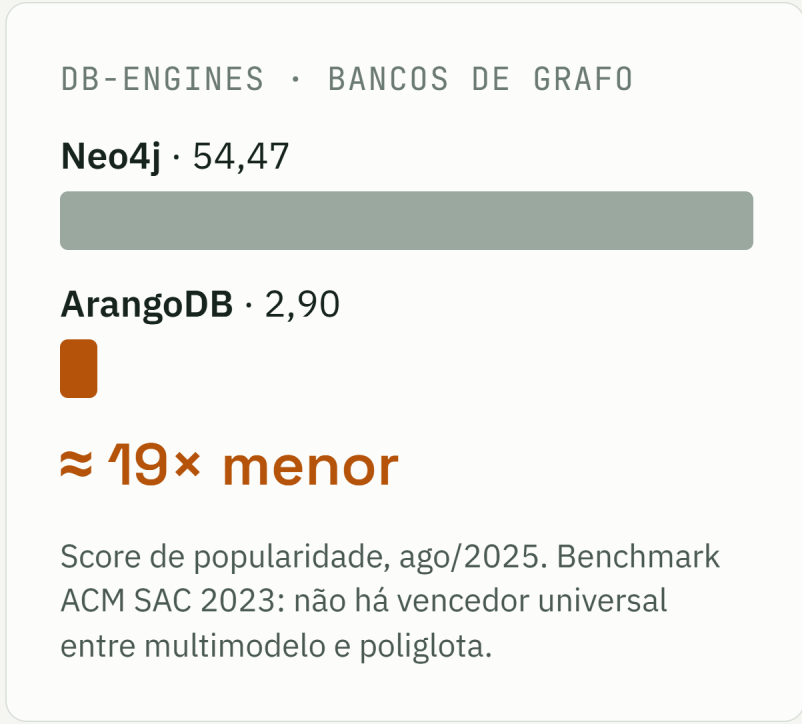
Não é o especialista em cada modelo: faltam recursos como os algoritmos de grafo do Neo4j ou a latência em memória do Redis.

Comunidade menor: menos tutoriais, respostas e vagas pedindo a ferramenta.

Curva da AQL: não é SQL, nem Cypher, nem a API do MongoDB.

Modelagem mais complexa: a equipe decide o que é documento, aresta ou chave.

Um fornecedor só: toda a persistência depende de um único motor.



PARTE 2 · VANTAGENS E DESVANTAGENS

A questão da licença

Desde a versão 3.12, o ArangoDB é “source-available”, não open source.

out/2023

Anúncio da mudança

Até aqui, a Community era Apache 2.0.

2024 · versão 3.12

Código vai para BSL 1.1

Licença que a OSI não reconhece como open source.

versão 3.12.5

Community ganha tudo da Enterprise

Por isso o nosso painel mostra “Enterprise Edition”.

Gratuita, com todos os recursos

Proibido uso comercial em produção

Limite de 100 GiB de dados

Cada versão vira Apache 2.0 após 4 anos

PARTE 2 · CASOS DE USO

Quem usa o ArangoDB

Cenário	Empresa	O que fez
Recomendação	About You	Recomendações personalizadas em escala
Visão única de dados	Refinitiv / Thomson Reuters	Plataforma “Single View of Everything”
Investigação	Kaseware	Gestão de casos investigativos
Catálogo	Key2Publish	1 milhão de produtos, busca em milissegundos
Redes sociais	Kode Solutions	Análise semântica de tweets com grafos
Mercado financeiro	Stockbit	Migrou do MySQL; consultas mais rápidas
Saúde e pesquisa	University of Oxford	Reduzir idas ao hospital e custos

O padrão é o do nosso projeto: entidades ricas (documentos) ligadas por muitos relacionamentos (grafo).

Coleções e documentos

Tudo é documento JSON; o grafo nasce de um tipo especial de coleção, a de arestas.

```
// documento em usuarios
{ "_key": "u01",
  "_id": "usuarios/u01",
  "_rev": "_hX3k...",
  "nome": "Arthur" }
```

```
// aresta em amizades
{ "_from": "usuarios/u01",
  "_to": "usuarios/u02",
  "desde": 2021 }
```

Campo	O que guarda	Quem define
_key	Chave única na coleção	Você ou o banco
_id	coleção/chave, único no banco	O banco
_rev	Versão do documento	O banco, a cada alteração
_from / _to	Vértices de origem e destino	Você, só em arestas

_from e _to já vêm indexados: o **edge index** é o que deixa a travessia de grafo rápida.

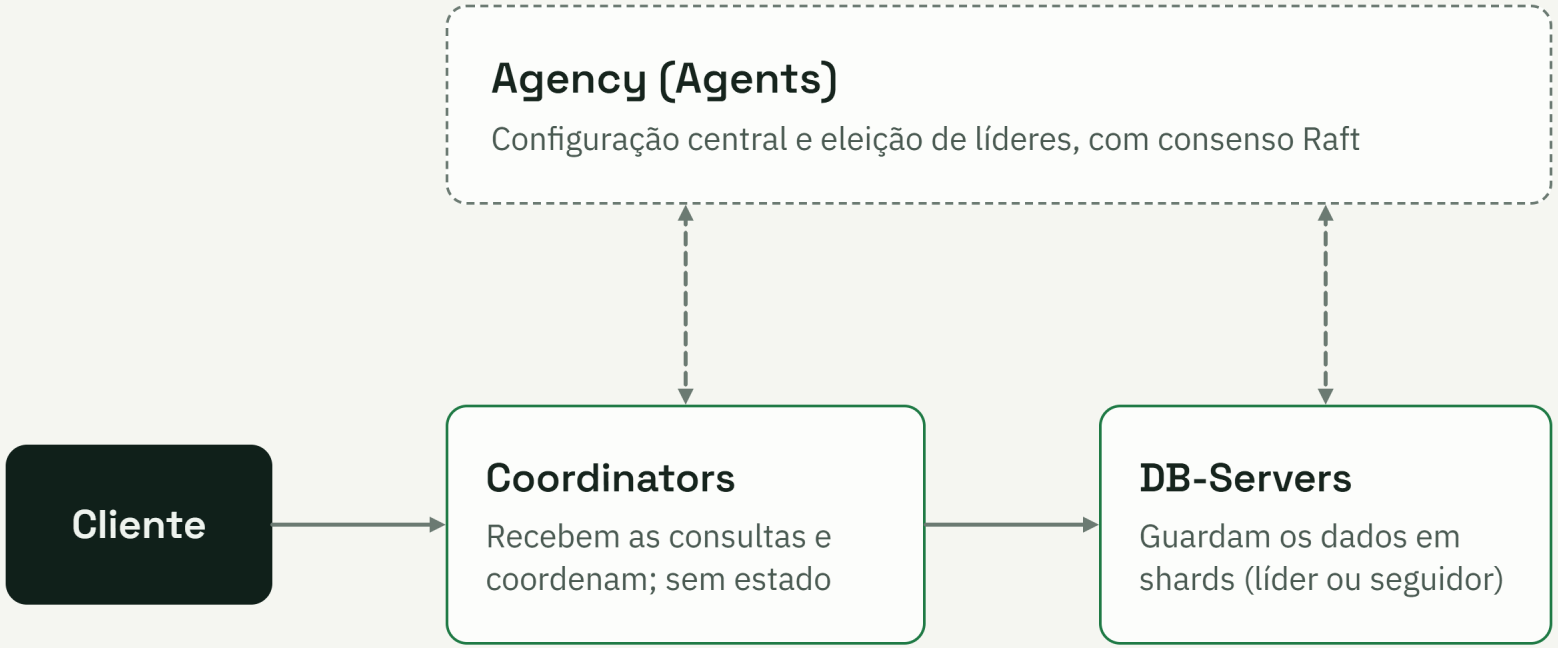
Servidor único x cluster

Servidor único

Um processo só, simples de subir. É o modo da nossa demo.

Cluster

Três papéis, modelo CP, sem ponto único de falha.



Shards e replication factor

Shards dão escalabilidade horizontal; o replication factor dá alta disponibilidade.

Conceito	Para que serve	Padrão
numberOfShards	Dividir a coleção entre máquinas	1 shard
shardKeys	Atributo do hash que escolhe o shard	_key
replicationFactor	Total de cópias de cada shard	1 (sem cópia)

```
db._create("usuarios", {
  numberOfShards: 4,
  replicationFactor: 3 })
```

replicationFactor 3

1 líder + 2 seguidores: o dado existe três vezes. Se o líder cai, um seguidor assume.

OneShard para grafos

Cada coleção num shard só, no mesmo DB-Server: a travessia não vai e volta pela rede.

AQL: parecida com SQL, escrita como laço

```
FOR u IN usuarios
  FILTER u.cidade == "Recife"
  SORT u.nome ASC
  LIMIT 5
  RETURN { nome: u.nome, idade: u.idade }
```

```
// JOIN = um FOR dentro de outro
FOR s IN sessoes
  FOR u IN usuarios
    FILTER u._id == s.usuario
    RETURN { nome: u.nome, sessao: s._key }
```

SQL	AQL
SELECT … FROM	FOR … IN … RETURN
WHERE	FILTER
ORDER BY	SORT
LIMIT	LIMIT
JOIN	FOR dentro de FOR
GROUP BY	COLLECT
INSERT INTO	INSERT … INTO
UPDATE … SET	UPDATE … WITH … IN
DELETE FROM	REMOVE … IN

Travessia de grafo em AQL

“Amigos de amigos” vira uma linha; no SQL seriam dois JOINs na tabela N:N.

```
FOR v, e, p IN 1..2 ANY "usuarios/u01" amizades
  FILTER v._key ≠ "u01"
  RETURN DISTINCT v.nome
```

Parte	Significado
v, e, p	Vértice, aresta e caminho
1..2	Profundidade mínima e máxima
OUTBOUND / INBOUND / ANY	Saída, entrada ou as duas
"usuarios/u01"	Vértice de partida, pelo _id
amizades	Coleção de arestas percorrida

Mais fundo?

Troque 1..2 por 1..3. No SQL, seria mais um JOIN ou uma CTE recursiva.

Modelar a partir das consultas

A pergunta deixa de ser “quais tabelas e chaves?” e vira “o que preciso buscar, e com que frequência?”.

Aspecto	Relacional (PostgreSQL)	ArangoDB (multimodelo)
Perfil	Colunas fixas; campo novo exige ALTER TABLE	Documento JSON; cada usuário pode ter campos diferentes
Amizade	Tabela N:N com duas chaves estrangeiras	Aresta com propriedades próprias
Amigos de amigos	Um JOIN a mais para cada nível	Travessia nativa, profundidade configurável
Sessão	Tabela comum, sem otimização	Acesso direto pela chave
Esquema	Definido antes de inserir	Flexível: vem dos próprios dados

PARTE 4 · MODELAGEM E INGESTÃO

Três coleções, três modelos

DOCUMENTO

usuarios

Perfil flexível: hobbies é uma lista embutida.

```
{ "_key": "u01",
  "nome": "Arthur",
  "cidade": "Recife",
  "hobbies": [
    "tecnologia",
    "futebol", "anime"] }
```

GRAFO · EDGE

amizades

O relacionamento é o dado; “desde” pertence à amizade.

```
{ "_from":
    "usuarios/u01",
  "_to":
    "usuarios/u02",
  "desde": 2021 }
```

CHAVE-VALOR

sessoes

Dado simples e temporário, lido direto pela _key.

```
{ "_key":
    "sessao_1001",
  "usuario":
    "usuarios/u01",
  "criada_em": "…" }
```

PARTE 4 · MODELAGEM E INGESTÃO

Embutir ou referenciar?

Embutir

Quando o dado pertence ao documento e é lido junto com ele.

Ex.: hobbies dentro do usuário

Referenciar

Quando a relação é N:N, cresce sem limite ou é consultada sozinha.

Ex.: amizades como arestas

Uma lista de amigos dentro do perfil até funciona, mas trava travessias de vários níveis, caminho mais curto e propriedades da relação. **É aí que o grafo se paga.**

Ingestão: do JSON às coleções

1. **Criar as coleções com o tipo certo:** amizades como Edge. O tipo não muda depois.
2. **Importar os arquivos:** pela interface web (Import JSON) ou pelo terminal, com arangoimport.
3. **Usuários antes das amizades:** senão ficam arestas soltas, apontando para quem não existe.
4. **Validar a carga** com uma consulta de contagem.

```
arangoimport --file usuarios.json \
  --type json --collection usuarios
arangoimport --file amizades.json \
  --type json --collection amizades
```

```
RETURN {
  usuarios: LENGTH(usuarios),
  amizades: LENGTH(amizades),
  sessoes: LENGTH(sessoes) }
```

20 usuários	**30** amizades	**0** sessões

CONCLUSÃO

Quando usar um banco multimodelo

Vale a pena

- Dados de formatos diferentes que se relacionam muito: rede social, recomendação, fraude
- Time pequeno
- Infraestrutura simples, com um banco só

Não vale a pena

- Máximo desempenho num modelo só: cache pesado no Redis, séries temporais no Cassandra
- Dados só tabulares e transacionais: relacional
- Quando uma comunidade grande é essencial

PARTE 5 ·

Demonstração

A rede social rodando no ArangoDB: 20 usuários, 30 amizades e os três modelos na mesma consulta.

1

Coleções e grafo
O visualizador da rede

2

CRUD
Criar, ler, atualizar e apagar o Zeca

3

Amigos de amigos
Grafo + documento numa query

4

Sugestão e caminho
Amigos em comum, menor caminho

5

Três modelos
Sessão → perfil → amigos

REFERÊNCIAS

Fontes consultadas

Documentação oficial do ArangoDB

- Cluster: arquitetura, papéis, sharding e replicação
- OneShard
- Índices: primary index e edge index
- Atributos de sistema dos documentos
- Travessias de grafo em AQL
- Edições Community x Enterprise e novidades da 3.12
- Anúncio e FAQ da mudança de licença; página de download

Estudos e análises

- ACM SAC 2023: multimodelo x poliglota (benchmark)
- DB-Engines: ranking de bancos de grafo
- phoenixNAP, BigBear.ai, Bloor Research, Tutorialspoint
- CaseStudies.com e FeaturedCustomers: estudos de caso
- Wikipedia: ArangoDB
- ArcadeDB: mudanças de licença no mercado
- Martin Fowler: persistência poliglota